Training Course

Linux Fundamentals

What is SHELL?

Linux Fundamental

- Bash
- Dash
- KornShell
- tcsh
- Zshell

What is Bash scripting ?

Example:

```
#!/usr/bin/env bash

name="John"
echo "Hello $name!"
```

Linux Fundamental

***Note**:* In the process of working with shell and bash scripting, if there is any command you don't understand or don't know, you SHOULD use '**man [command]'** to search for the definition and the usage of that command.

Note: In order for the script to run, you must grant the script file execute permission (‘x’ symbol if you run `ls -l`).

Variables

```
name="John"
echo $name # see below
echo "$name"
echo "${name}!"
```

Brace expansion

```
$ echo {A,B}.js
```

Expression	Result
{A,B}	Same as A, B
{A,B}.js	Same as A.js, B.js
{1..5}	Same as 1 2 3 4 5

Parameter expansion (basic)

```
name="John“
```

What can we do with $name

without piping to other commands?

Parameter expansions (basic)

Command	Result	Explanation
echo "${name}"	John	Normal behavior
echo "${name/J/j}"	john	substitution
echo "${name:0:2}"	Jo	Slicing except for Xth to Yth (from X pos
echo "${name::2}"	Jo	Slicing except for Xth to Yth (from X pos
echo "${name::-1}"	Joh	Slicing
echo "${name:(-1)}"	n	Slicing from right
echo "${name:(-2):1}"	h	Slicing from right
echo "${food:-Cake}"	Cake	If $food isn't set then set it to food="Cake"

Loops

Type	Example
Basic loop	for i in /etc/rc.*; do echo "$i" done
C-like loop	for ((i = 0 ; i < 100 ; i++)); do echo "$i" done

Loops

Type	Example
Basic range	for i in {1..5}; do echo "Welcome $i" done
Range with steps	for i in {5..50..5}; do echo "Welcome $i" done
Reading lines	while read -r line; do echo "$line" done <file.txt

Linux Fundamentals

- **Switch case**

```
case EXPRESSION in
  PATTERN_1)
    STATEMENTS
    ;;
  PATTERN_2)
    STATEMENTS
    ;;
  PATTERN_N)
    STATEMENTS
    ;;
  *)
    STATEMENTS
    ;;
esac
```

Linux Fundamentals

- **Switch case (example)**

```
case "$1" in
start | up)
vagrant up
;;
*)
echo "Usage: $0 {start|stop|ssh}"
;;
esac
```

Note: to get the input from the terminal and use it in the script, you can you command read. For example:

```
echo -n "Proceed? [y/n]: "
read -r ans
echo "$ans"
```

Functions

- Defining functions

	Example

Simple way	myfunc() {
	  echo "hello $1"
	}

Alternate sytax	function myfunc() {
	  echo "hello $1"
	}

Function

- Returning values

```
myfunc() {
    local myresult='some value'
    echo "$myresult"
}
result=$(myfunc)
```

Function

- Rasing errors

```
myfunc() {
  return 1
}

if myfunc; then
  echo "success"
else
  echo "failure"
fi
```

Function

- Arguments

Arguments	Definition
$#	Number of arguments
$*	All positional arguments (as a single word)
$@	All positional arguments (as separate strings)
$1	First argument
$_	Last argument of the previous command

Note: [[is actually a command/program that returns either 0 (true) or 1 (false). Any program that obeys the same logic (like all base utils, such as grep(1) or ping(1)) can be used as condition, see examples.

Conditionals

Expression	Definition
[[-z STRING]]	Empty string
[[-n STRING]]	Not empty string
[[STRING == STRING]]	Equal
[[STRING != STRING]]	Not equal
[[STRING =~ STRING]]	Regexp

Conditionals

Expression	Definition
[[NUM -eq NUM]]	Equal
[[NUM -ne NUM]]	Not equal
[[NUM -lt NUM]]	Less than
[[NUM -le NUM]]	Less than or equal
[[NUM -gt NUM]]	Greater than
[[NUM -ge NUM]]	Greater than or equal

Conditionals

Expression	Definition
((NUM == NUM))	Equal
((NUM != NUM))	Not equal
((NUM < NUM))	Less than
((NUM <= NUM))	Less than or equal
((NUM > NUM))	Greater than
((NUM >= NUM))	Greater than or equal

Conditionals

Expression	Definition
[[-o noclobber]]	If OPTIONNAME is enabled
[[! EXPR]]	NOT
[[X && Y]]	AND
[[X || Y]]	OR

Condictions' example

Type	Example
String compare	if [[-z "$string"]]; then echo "String is empty" elif [[-n "$string"]]; then echo "String is not empty" else echo "This never happens" fi

Regex	if [[$STRING =~ .*$STR2FIND.*]]; then
	echo "The string exists in the text."
	else
	echo "The string doesn't exist in the text."
	fi

File condictions

Expression	Definition
[[-e FILE]]	File exist?
[[-r FILE]]	File is readable?
[[-h FILE]]	Symlink?
[[-d FILE]]	Directory?
[[-w FILE]]	File is writable?
[[-s FILE]]	File’s size > 0 byte?
[[-f FILE]]	File?

File condictions

Expression	Definition
[[-x FILE]]	File is executable?
[[FILE1 -nt FILE2]]	File 1 is opened more recently than File 2
[[FILE1 -ot FILE2]]	File 1 is opened less recently than File 2
[[FILE1 -ef FILE2]]	File 1 is the same as File 2

- **Dealing with outputs**

	Example

Redirecting output	$ date > today.txt
	$ cat today.txt
	Sun Jul 23 13:52:57 +07 2023

Appending output	$ who >> today.txt
	$ cat today.txt
	Sun Jul 23 13:52:57 +07 2023
	thanhhv18 pts/1 2023-07-23 13:38

- **Dealing with outputs**

	Example
Pipiling data	$ ls | sort Desktop Documents Downloads test.txt today.txt Videos

Linux Fundamentals

- **Arrays**

```
#!/usr/bin/env bash
Fruits=('Apple' 'Banana' 'Orange')
```

```
#Logic
Fruits[0]="Apple"
Fruits[1]="Banana"
Fruits[2]="Orange"
```

- **Arrays**

Command	Definition
echo "${Fruits[0]}"	Element #0
echo "${Fruits[-1]}"	Last element
echo "${Fruits[@]}"	All elements, space-separated
echo "${#Fruits[@]}"	Number of elements
echo "${#Fruits}"	String length of the 1st element
echo "${#Fruits[3]}"	String length of the Nth element
echo "${Fruits[@]:3:2}"	Range (from position 3, length 2)
echo "${!Fruits[@]}"	Keys of all elements, space-separated

- **Arrays**

Fruits=("${Fruits[@]}" "Watermelon")	Push
Fruits+=('Watermelon')	Also push
Fruits=("${Fruits[@]/Ap*/}")	Remove by regex match
unset Fruits[2]	Remove 1 item
Fruits=("${Fruits[@]}")	Duplicate
Fruits=("${Fruits[@]}" "${Veggies[@]}")	Concatenate

Self study

- **Getting user's options (advanced)**

```
while [[ "$1" =~ ^- && ! "$1" == "--" ]]; do case $1 in
  -V | --version )
    echo "$version"
    exit
    ;;
  -s | --string )
    shift; string=$1
    ;;
  -f | --flag )
    flag=1
    ;;
esac; shift; done
if [[ "$1" == '--' ]]; then shift; fi
```

- **Special variables**

$?	Exit status of last task
$!	PID of last background task
$$	PID of shell
$0	Filename of the shell script
$_	Last argument of the previous command
${PIPESTATUS[n]}	Return value of piped commands (array)

Thank you